

Robotics in Go

Building a Raspberry Pi Rover

Tristan Penman

Melbourne Go Meetup · May 14, 2026

Agenda

5 mins

Demo & Hardware Tour

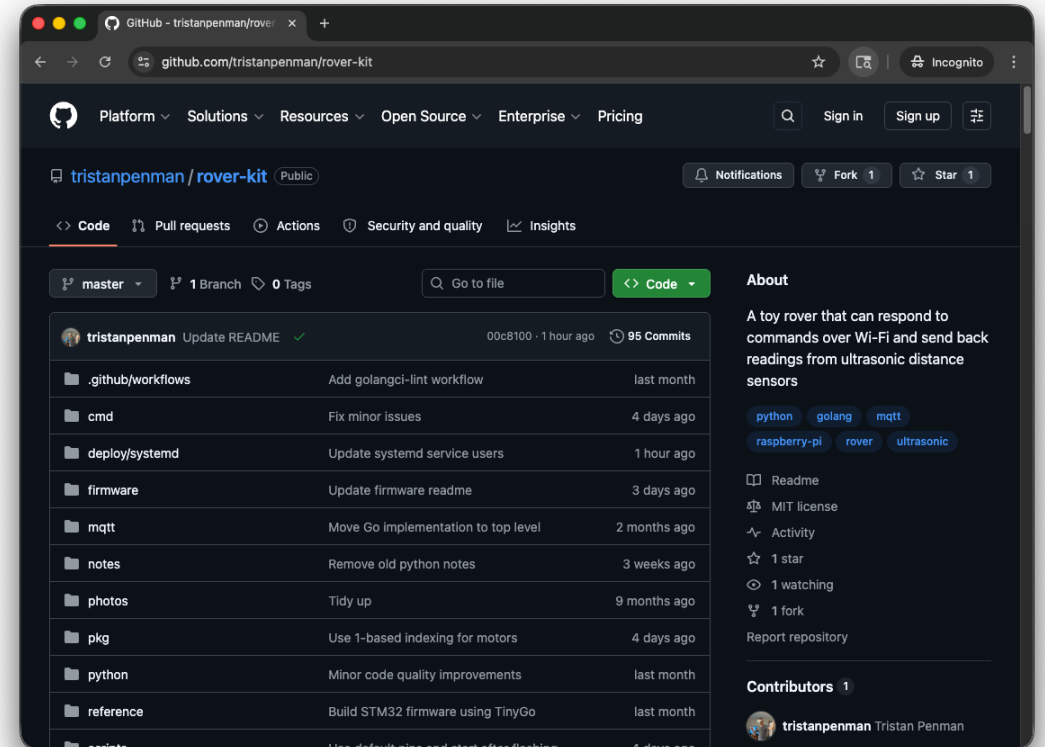
10 mins

Robotics, Hardware Interfaces and Go

10 mins

Building Components with MQTT

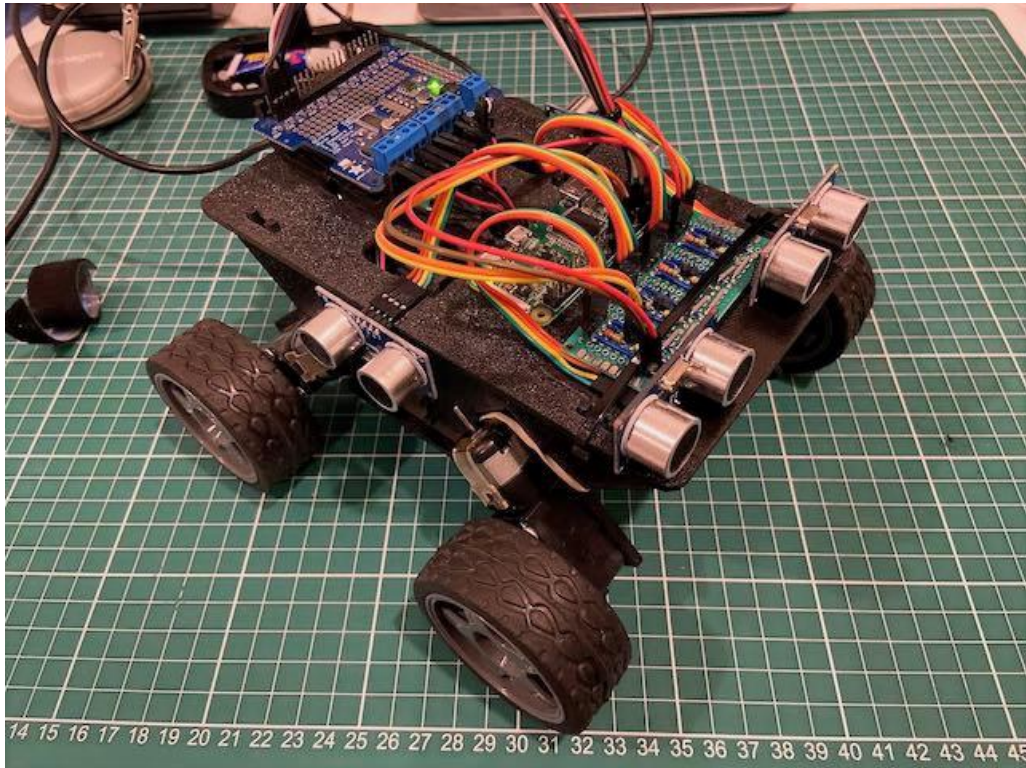
Source code: github.com/tristanpenman/rover-kit



Demo

Let's drive

The Hardware



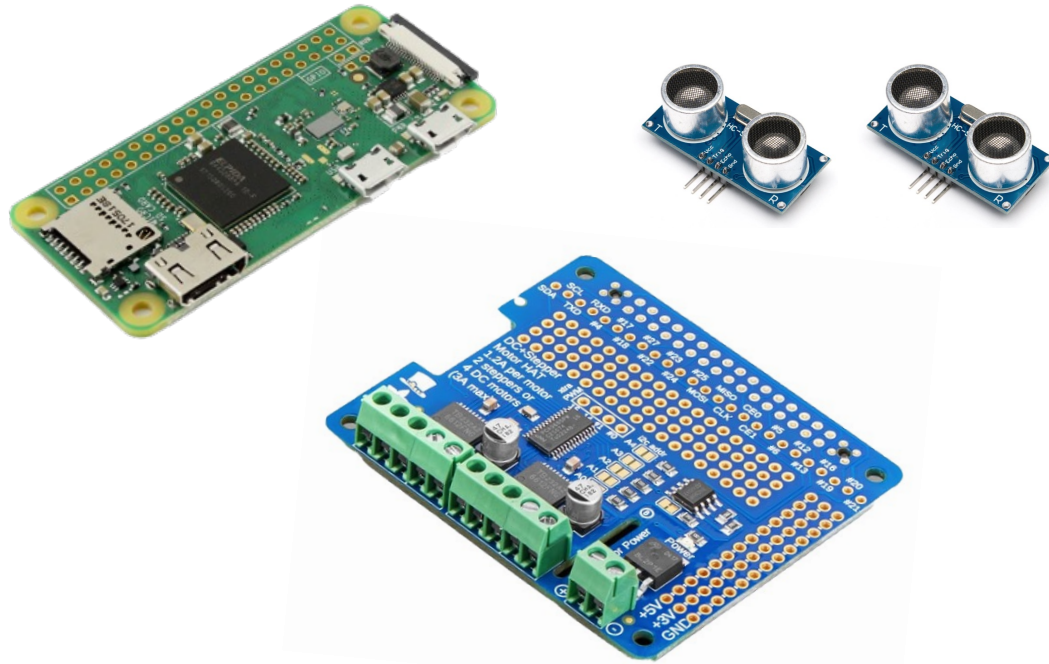
Bill of materials

- Raspberry Pi Zero W (ARMv6)
- 4× DC motors via Adafruit Motor HAT (I²C)
- 4× HC-SR04 ultrasonic sensors (GPIO)
- Whippersnapper Runt Rover chassis
- 12V battery for motors
- 5V USB power supply for the Raspberry Pi

No SLAM, no inverse kinematics, and no AI.

Just enough hardware to explore the software stack.

Individual Components



Pi Zero W

Brains of the operation. Runs Go binaries and MQTT

Motor HAT

I²C-driven PWM controller (PCA9685)

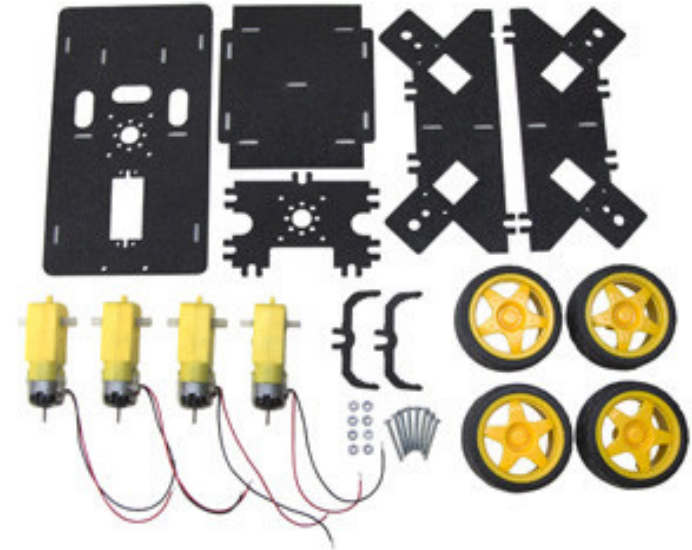
Sonar array

HC-SR04 ultrasonic sensors, mounted around chassis

Whippersnapper Runt Rover Kit

Includes 4x DC right-angle motors

Whippersnapper Runt Rover



Robotics Fundamentals

Actuation

Controlling joints and motors.

The rover has four DC motors for actuation. Each motor is independently controlled, on a fixed axis, making this a **Differential Drive**.

Sensors

Observing the environment.

Examples include cameras, IR sensors, pressure and temperature sensors. The rover includes ultrasonic sensors for measuring distance to the nearest obstacle.

Navigation

Manoeuvring in the world.

Known as “localization” and “mapping”. This iteration of the rover doesn’t know where it is in the world.

Once you go further:

- Real-time control — precise timing constraints
- Embedded targets — low power, harsh environments
- Coordinated machinery — many joints, high energy
- Safety & critical infrastructure

Robot Operating System



The default choice in research and industry. ROS provides:

Middleware

Pub/sub message passing infrastructure.

Enables loose coupling and component isolation.

Drivers

Sensors, motor controllers, SLAM, path planning.

Wide hardware support and a huge ecosystem make ROS dominant in research and prototyping.

Standard messages

Geometry, sensors, navigation metadata, task planning, etc.

Tooling

Visualisation, introspection and debugging.

This is a strength of ROS. Simulation is often how projects start, allowing hobbyists to learn with simulated versions of expensive hardware.

Frustrations with ROS*

ROS 1 vs 2

- Choosing between ROS 1 and 2...
- ROS 1 was designed for research, with a central control node.
- ROS 2 attempts to improve support for commercial applications.
- ROS 1 reached end-of-life in 2025, which means a lot of resources online are outdated.

Architecture Support

- Compiling ROS 1 for ARMv6 seemed unnecessarily difficult. Not worth the time for an EOL release.
- Pre-built ROS 1 packages tied to specific OS releases due to deep dependency trees.

Development Experience

- While comprehensive, ROS can get in the way with smaller projects.
- Build system is... interesting.
- Inconvenient to work with ROS projects from within an IDE

** These are my personal frustrations*

Why Go for Robotics?

Concurrency

Goroutines + channels map cleanly to async sensor/actuator loops

Memory safety

No pointer arithmetic, memory-safe runtime

Simplicity

Often easier to read than C++ or Python

Cross-compilation

Self-contained binaries, minimal runtime dependencies

Iteration

Fast builds, simple deps, good testing support

Concurrency Model

Activities that are conceptually independent but must coordinate.

You might assume that the rover software is monolithic, with each component managed by its own goroutine. The reality is that process isolation provides many benefits.

However, goroutines provide clear and mature patterns within individual components!

- A goroutine owns a long-running activity (e.g., sonar sampling)
- A channel becomes the handoff for produced values.
- Mutexes still serialise access to shared hardware state.
- **context.Context** provides a common set of shutdown/interrupt behaviours.

Goroutines are cheap. Motors, I²C buses, and physical momentum are not.

```
// Open returns a channel of Reading

ch := provider.Open(ctx)
for r := range ch {
    publish(ctx, r)
}

<-ctx.Done()
```

Cross-compilation & Caveats

Cross-compilation

Native ARMv6 builds for the Pi Zero W from your laptop:

```
GOOS=linux GOARCH=arm GOARM=6 \  
go build ./cmd/...
```

- Self-contained binaries, minimal runtime
- Binaries can be copied over to the Pi and run via systemd or independently.
- Communication between components is handled by MQTT.

Caveats: Garbage Collection

- GC can introduce unpredictable latency spikes
- Timing discrepancies hurt perception & coordination
- Some motor configs may overshoot or undershoot
- Mitigation: push timing-sensitive work off-board

Hardware Interfaces

GPIO

General-purpose digital pins. The building block for everything else.

I²C

Two-wire addressed bus. Used by the Adafruit Motor HAT (PCA9685).

SPI

Fast synchronous serial bus. Each chip selected by a dedicated pin.

UART

Asynchronous serial — perfect for talking to external microcontrollers.

I²C – *Inter-Integrated Circuit*

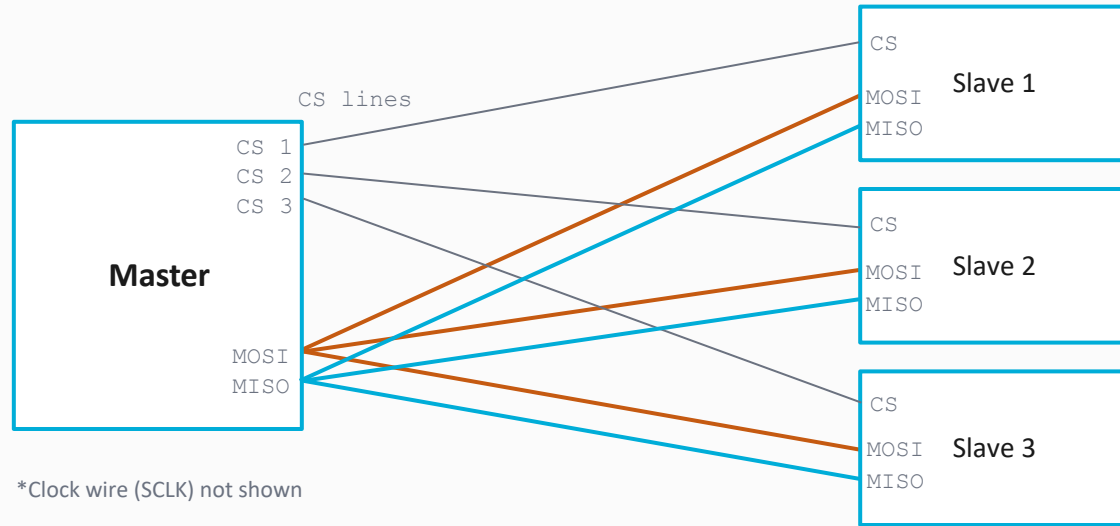


A single I²C transaction: address the chip, ack, send a byte, ack, stop.

Key points

- Two wires: SDA + SCL
- Multiple devices on one bus
- Devices use a 7-bit address
- PCA9685 PWM chip on the Motor HAT allows us to drive 4 motors using just 2 wires

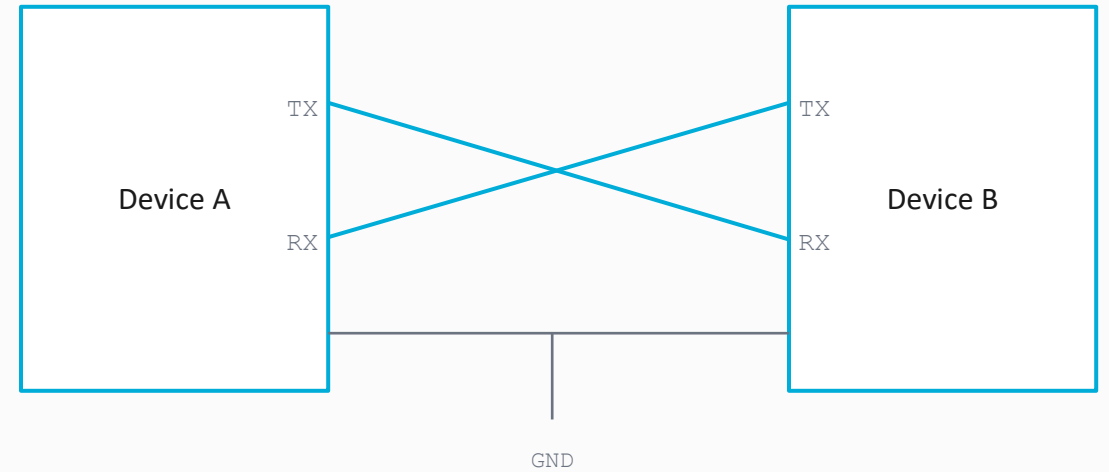
SPI and UART



SPI

Fast synchronous serial.

One master, multiple slaves selected by dedicated chip-select pins.



UART

Async serial — TX, RX, GND. Great for talking to external MCUs.

USART adds a clock for higher speeds.

The Go Hardware Ecosystem

Gobot

`gobot.io`

Robotics framework, with support for a range of devices, drivers, adaptors across many SBCs.

Periph

`periph.io`

Low-level GPIO/I²C/SPI/PWM. Lower-level control.

gpiocdev

`warthog618/go-gpiocdev`

Native Go GPIO over the Linux character device.

go.bug.st/serial

`bugst/go-serial`

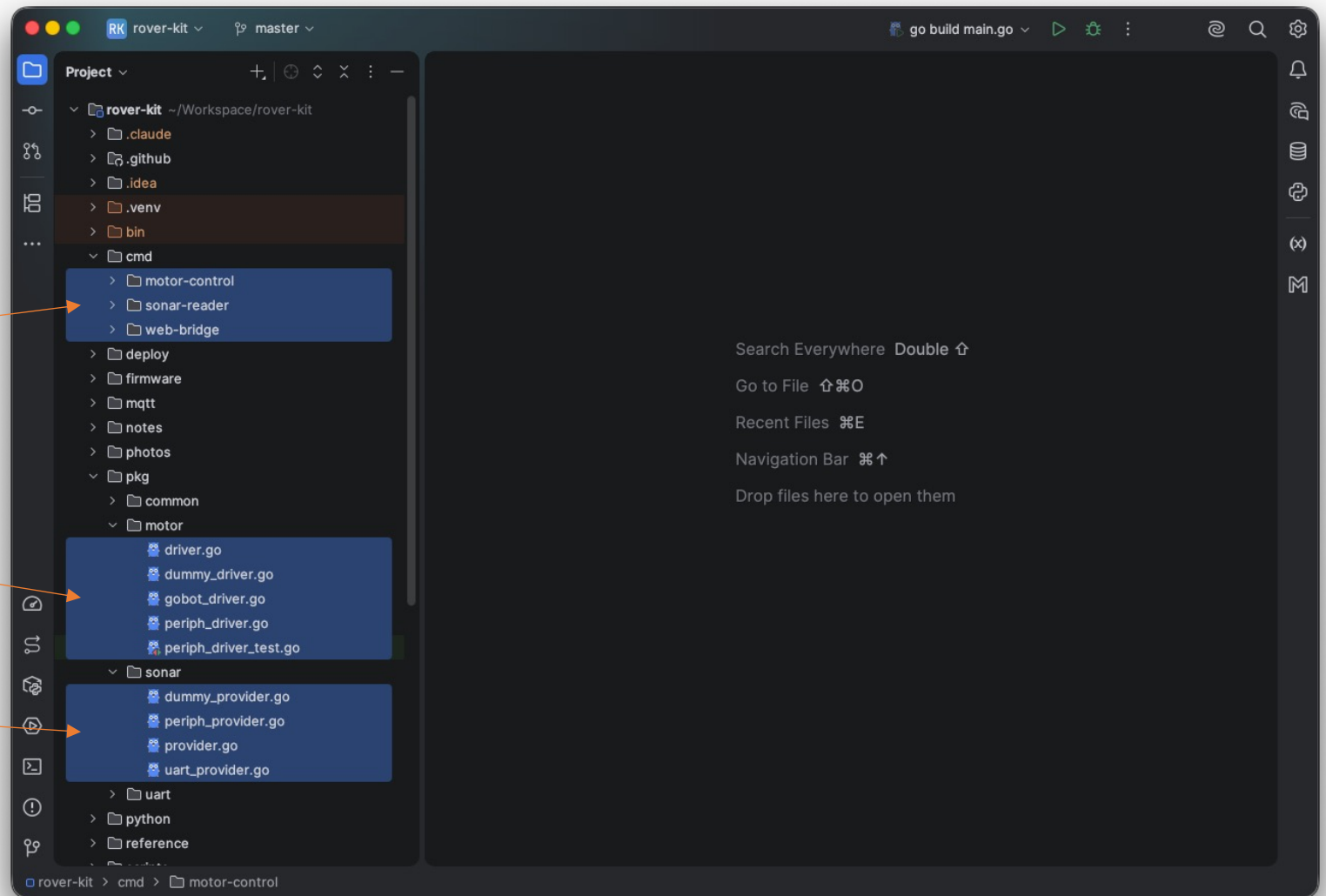
Cross-platform serial port library.

Rover Kit

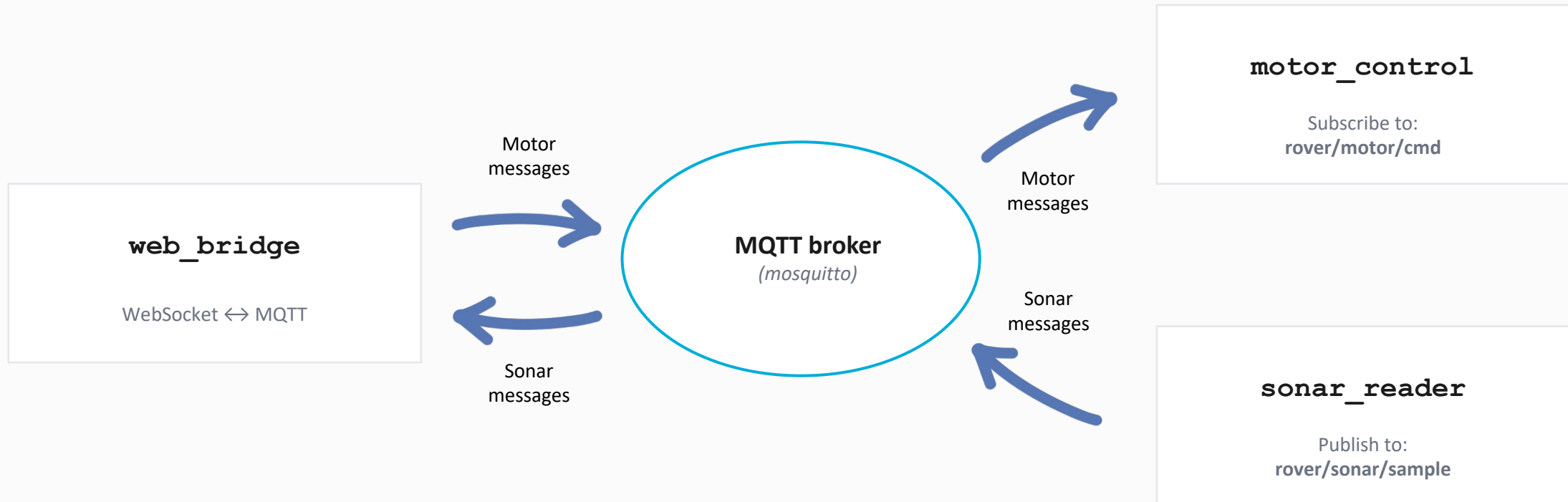
Three commands

Motor drivers

Sonar providers



Building Components with MQTT



Each binary runs under systemd on the Pi and only knows about the broker.

Messages can be sent over the network for fast development.

Eclipse Paho MQTT Client

All we really need to know are three basic operations:

Connect

```
client := mqtt.NewClient(opts)
t := client.Connect()
t.Wait()
if err := t.Error(); err != nil {
    log.Fatal(err)
}
```

Subscribe

```
client.Subscribe(
    motorCmdTopic,
    1,
    subscriber(ctx, drv),
)
```

Publish

```
s.mqttClient.Publish(
    defaultMotorCmdTopic,
    0, false, message,
)
```

Motor Control & Sonar Reader

motor_control

Subscribes to **rover/motor/cmd**.

Drives 4× DC motors via the Adafruit Motor HAT.

```
// Driver interface implements these operations:
```

```
type Driver interface {  
    Forwards(ctx) error  
    Backwards(ctx) error  
    SpinCW(ctx) error  
    SpinCCW(ctx) error  
    Stop(ctx) error  
}
```

```
// Gobot or Periph providers
```

sonar_reader

Samples HC-SR04 sensors.

Publishes values to **rover/sonar/sample**.

```
// Provider returns a channel of Reading:
```

```
type Reading struct {  
    DistanceCM float64  
    DurationUS float64  
    Timestamp  time.Time  
}
```

```
type Provider interface {  
    Open(ctx context.Context) chan Reading  
}
```

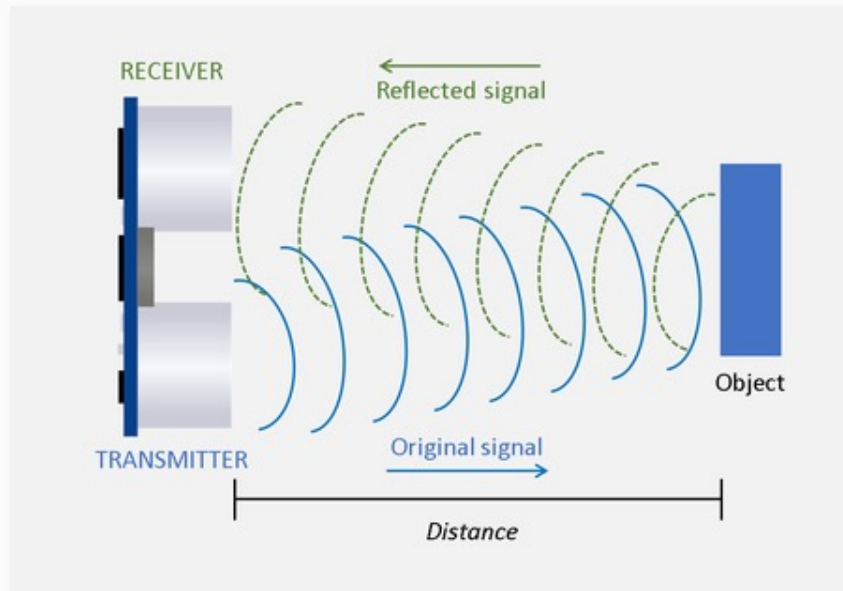
```
// Periph or UART providers
```

Sonar Provider: Initialisation



```
func NewPeriphProvider() (*PeriphProvider, error) {  
    if _, err := host.Init(); err != nil { return nil, fmt.Errorf("initialize periph host: %w", err) }  
  
    trig := gpioreg.ByName(defaultTriggerPin)  
    if trig == nil { return nil, fmt.Errorf("trigger pin %s not found", defaultTriggerPin) }  
  
    echo := gpioreg.ByName(defaultEchoPin)  
    if echo == nil { return nil, fmt.Errorf("echo pin %s not found", defaultEchoPin) }  
  
    // no pull on echo pin  
    err := echo.In(gpio.Float, gpio.NoEdge)  
    if err != nil { return nil, err }  
  
    // default trigger pin to low  
    err = trig.Out(gpio.Low)  
    if err != nil { return nil, err }  
  
    return &PeriphProvider{...}, nil  
}
```

Sonar Provider: Loop



```
func (p *PeriphProvider) Open(context.Context) chan Reading {
    c := make(chan Reading)

    go func() {
        defer close(c)

        for {
            // set trigger pin high
            err := p.trig.Out(gpio.High)
            if err != nil {...}

            // sleep for `triggerPulse` nanoseconds
            time.Sleep(triggerPulse)

            // set trigger low
            err = p.trig.Out(gpio.Low)
            if err != nil {...}

            start := time.Now()

            // wait for echo high
            timedOut := false
            for p.echo.Read() != gpio.High {...}
            if timedOut {...}

            start = time.Now()

            // wait for echo low
            for p.echo.Read() != gpio.Low {...}
            if timedOut {...}

            end := time.Now()

            duration := end.Sub(start)

            c <- Reading{...}

            time.Sleep(sampleInterval)
        }
    }()

    return c
}
```

Motor Drivers

Gobot

```
func NewGobotDriverWithThreshold(threshold float64) (*GobotDriver, error) {
    adaptor := raspi.NewAdaptor()
    if err := adaptor.Connect(); err != nil {
        return nil, fmt.Errorf("connect raspi adaptor: %w", err)
    }

    hat := i2c.NewAdafruitMotorHatDriver(
        adaptor,
        i2c.WithBus(1),
        i2c.WithAddress(defaultGobotMotorHatI2C),
    )
    if err := hat.Start(); err != nil {
        _ = adaptor.Finalize()
        return nil, fmt.Errorf("start Adafruit motor hat driver: %w", err)
    }

    driver := &GobotDriver{
        adaptor:  adaptor,
        hat:       hat,
        threshold: threshold,
    }
    if err := driver.Stop(context.Background()); err != nil {
        _ = hat.Halt()
        _ = adaptor.Finalize()
        return nil, fmt.Errorf("stop motors during startup: %w", err)
    }

    return driver, nil
}
```

Periph

```
func NewPeriphDriverWithThreshold(threshold float64) (*PeriphDriver, error) {
    if _, err := host.Init(); err != nil {
        return nil, fmt.Errorf("initialize periph host: %w", err)
    }

    bus, err := i2creg.Open("")
    if err != nil {
        return nil, fmt.Errorf("open i2c bus: %w", err)
    }

    dev, err := pca9685.NewI2C(bus, defaultPeriphMotorHatI2C)
    if err != nil {
        _ = bus.Close()
        return nil, fmt.Errorf("initialize pca9685 on i2c address 0x%X: %w",
            defaultPeriphMotorHatI2C, err)
    }

    if err := dev.SetPwmFreq(1600); err != nil {
        _ = bus.Close()
        return nil, fmt.Errorf("set pca9685 pwm frequency: %w", err)
    }

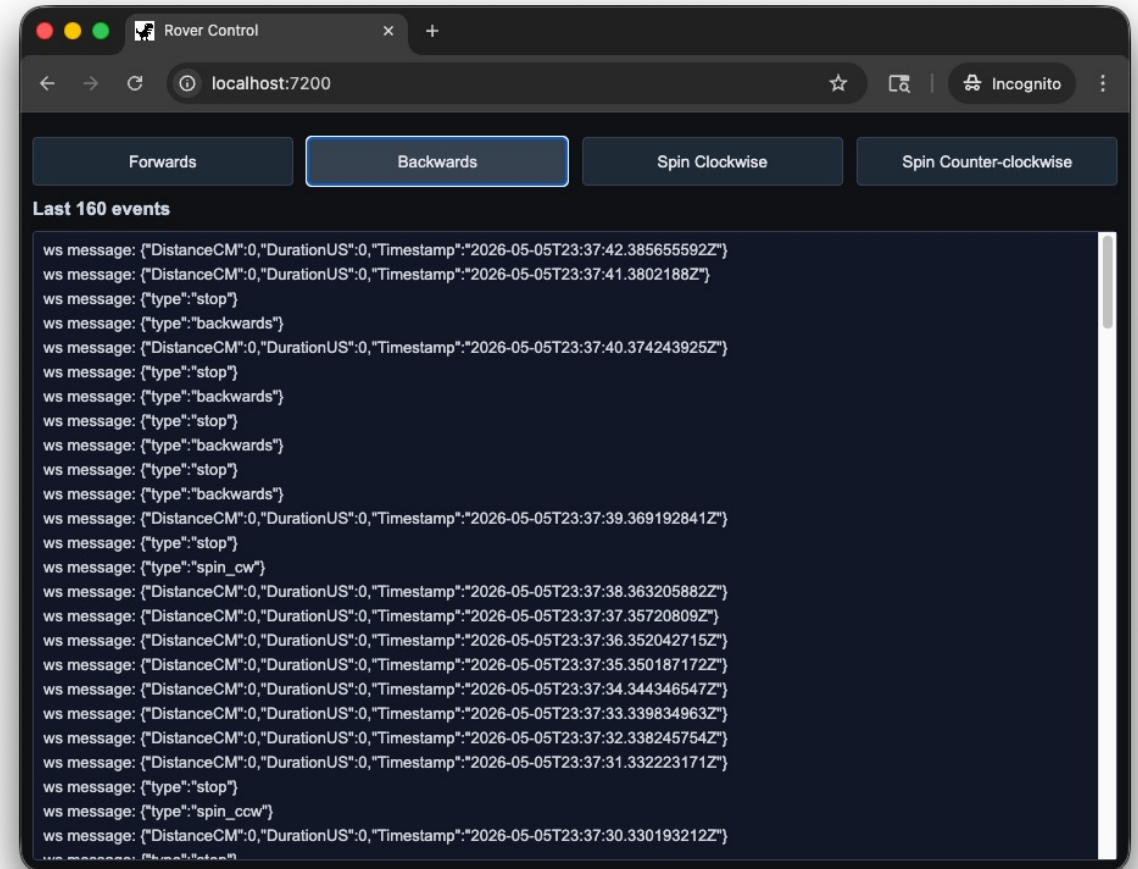
    driver := &PeriphDriver{bus: bus, pca: dev, threshold: threshold}
    if err := driver.Stop(context.Background()); err != nil {
        _ = bus.Close()
        return nil, fmt.Errorf("stop motors during startup: %w", err)
    }

    return driver, nil
}
```

Web Bridge

- Tiny static site (HTML, CSS, vanilla JS)
- Browser opens a WebSocket to `/ws`
- Validated commands → `rover/motor/cmd`
- Subscribes to `rover/sonar/sample`
- Fans sonar readings out to every connected client

```
{"type": "forwards"}  
{"type": "spin_ccw"}  
{"type": "stop"}
```



Next Steps

Off-board sonar

Move ultrasonic timing onto an STM32 — escape Go's GC for timing-sensitive work.

TinyGo on the STM32

Compile Go for the Cortex-M4. Much closer to real-time performance.

USB UART bridge

Stream distance readings from STM32 → Pi via go.bug.st/serial.

Better autonomy

Pub/sub semantics make new behaviours easy to drop in.

Thank you

Questions?

github.com/tristanpenman/rover-kit